

6 最短路与网络流：网络矩阵为什么不需要分支

第 5 章的匈牙利算法里，已经出现了“最短路加一组价格”。这一章把它放回一般的网络：最短路、最大流、最小费用流是同一族问题，它们的约束矩阵全单模，所以 LP 直接给整数解。再看两种让这个性质消失的加法：共用限额，和多个商品共用一张网络。

学完本章，你能

- 1. 说出 Dijkstra 依赖的那条性质，判断一张网络该用 Dijkstra、Bellman-Ford 还是 Johnson，会用势把负费用改写成非负。
- 2. 手写 Edmonds-Karp，解释残量图和回退弧，用最小割给最大流做证书。
- 3. 手写逐次最短路的最小费用流，说明费用曲线为什么是凸的，并把指派和运输问题写成网络。
- 4. 证明点弧关联矩阵全单模，判断给网络再加一行约束之后整数性还在不在；知道多商品流为什么需要分支。

本章信息

用时	阅读与交互 80-110 分钟，Lab 3 小时左右
前置	第 1、2、5 章（LP、对偶价格、约化成本、全单模）。会读 Python 代码。
对拍	SciPy 1.17.1 · HiGHS 1.12.0；本章的算法由页面内置代码实现，不依赖求解器。
配套	Lab 代码包：5 个练习 + 1 个挑战，含手写的最短路、最大流和最小费用流，pytest 自动判分

PART 1 数学规划基础	PART 2 组合优化	PART 3 调度理论	PART 4 路径与波次
PART 5 大规模方法	PART 6 启发式	PART 7 多智能体时空规划	PART 8 动态与不确定

第二部分 组合优化：指派问题与匈牙利算法 → 最短路与网络流（本章） → 带容量的指派与背包 → 旅行商与车辆路径入门。

6.1 最短路与 Dijkstra

第 5 章每插入一个任务，都在“约化成本”上跑一次 Dijkstra，然后涨价。当时把它当成匈牙利算法的一个零件。它其实是一个独立的、更一般的工具：在一张有向网络上，找一个点到所有点的最短路。

本章从一张 6 个点的小网络开始，它会在后面每一节出现：S 是源点，T 是汇点，a、b、c、d 是中间站。每条弧有两个数，容量和费用。这一节只用费用（图上弧旁边的数字），把它想成行驶时间。

最短路的问题很朴素：从 S 出发，到每个点的最短距离是多少，沿哪条路走？Dijkstra 的做法是给每个点维护一个标签（目前找到的最短距离），每一步取标签最小、还没有“确定”的点，把它确定下来，再用它去改进邻点的标签。

松弛 $d(v) \leftarrow \min(d(v), d(u) + c_{uv})$ 用点 u 的标签去试着改进邻点 v

数组版每一步扫描全部点找最小标签，共 n 步，总共 $O(n^2 + m)$ ；用堆可以降到 $O(m \log n)$ 。第 5 章的 `insert` 用的就是数组版。

算法能成立，靠的是一句话：取出来的点，它的标签已经是最终答案，以后不会再改。理由是，任何还没确定的点，标签都不小于当前这个点；从它们出发再走一段，只会更长。这句话有一个前提：走一段不会变短，也就是所有费用非负。

先预测 在原网络上多加一条弧 $c \rightarrow b$ ，费用 -4 （从 c 走到 b 反而省 4）。直接用 Dijkstra 求 S 到 T 的最短距离，会得到什么？

报错，因为有负费用

6，正确答案

7，比正确答案大 1，没有任何提示

陷入死循环

预测正确。 标准的 Dijkstra 不检查费用的符号。b 在 c 之前就被确定为 2，等 c 被确定（标签 5）， $c \rightarrow b$ 想把 b 改成 1，但 b 已经确定，不再回头，最终 $T = 7$ 。正确答案 6 要走 $S \rightarrow a \rightarrow c \rightarrow b \rightarrow d \rightarrow T$ 。下面的图已经解锁，切到“加一条 $c \rightarrow b$ (-4)”一步步看。

网络

原网络 (费用非负)

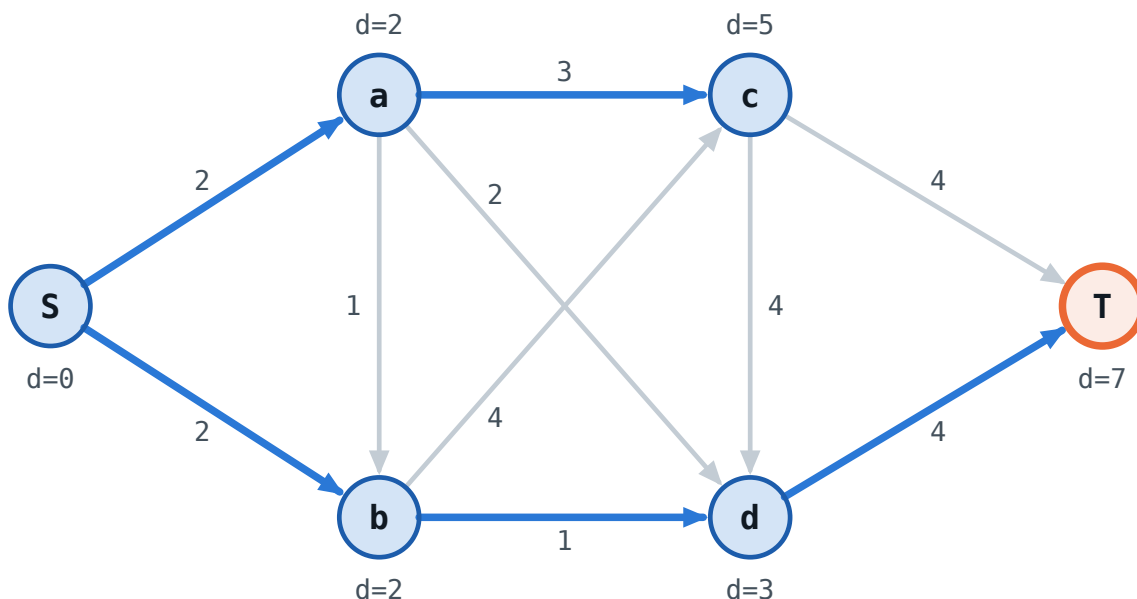
加一条 $c \rightarrow b$ (-4)把 $c \rightarrow b$ 改成 -6 (负环)

算法

Dijkstra

Bellman-Ford

Johnson



— 已确定的最短路树 — 这一步改进了标签 — 被跳过或在负环上

费用全部非负

S→T 最短距离 7

点	标签	正确值
S	0	0 已确定
a	2	2 已确定
b	2	2 已确定
c	5	5 已确定
d	3	3 已确定
T	7	7 已确定

确定 **T** (标签 7)。它没有能改进的邻点。

结束。 S 到 T 的最短距离是 7。

图 6-1 最短路。弧上的数字是费用，点旁边是当前标签。选算法，一步步看标签怎么被改进；换网络，看负费用弧和负环怎么让算法出错或报警。“正确值”一列来自 Bellman-Ford。

读图

先选“原网络”和 **Dijkstra**。S 确定后，a 和 b 的标签都变成 2；确定 b 时 d 得到 3；d 确定之后 T 得到 7。c 先从 a 得到 5，b→c 给的是 6，不比 5 小，所以没有变。最后 S 到 T 的最短距离是 7，路径 S→b→d→T。

再换成“加一条 **c→b (-4)**”。这条新弧从 c 回到 b，费用是负的：从 c 走到 b 不但不花时间，还“省”了 4。真正的最短路是 S→a→c→b→d→T，费用 $2 + 3 - 4 + 1 + 4 = 6$ 。Dijkstra 在确定 b 的时候，c 还没有出现；等 c 被确定，它想把 b 从 2 改成 1，但 b 已经确定了，Dijkstra 不回头。最后给出的是 7，比正确答案 6 多了 1，而且没有任何报错。

这种“悄悄给错”是这一节最想让你记住的事。用 Dijkstra 之前，先确认费用非负，Lab 练习 1 要求负费用时直接抛出异常。

6.2 负费用、势与 Johnson

费用真的可能是负的吗？在物流里比较少见，但把别的问题翻译成网络时经常出现：回程带货可以抵消一部分成本，最小费用流的残量图里，回退弧的费用就是原来的相反数。

Bellman-Ford：不依赖“确定”

Bellman-Ford 不做“确定”，每一轮把**所有弧**各松弛一遍。理由是，最短路至多有 $n - 1$ 条弧，所以 $n - 1$ 轮之后，所有标签都应该是最终值。代价是 $O(nm)$ ，比 Dijkstra 慢，但对费用的符号没有要求。

它还有一个副产品：**负环检测**。跑完 $n - 1$ 轮之后再松弛一轮，如果还有标签变小，说明有一个环越绕越短，最短路不存在。沿前驱指针回走 n 步，一定落在这个环上，把它读出来就是证据。在图 6-1 里选“把 c→b 改成 -6”，再切到 Bellman-Ford，跑到最后一步看看。

Johnson：用势把负费用改写成非负

如果要从很多个点出发求最短路，每次都跑 Bellman-Ford 太贵。Johnson 的想法是：给每个点一个数 $h(v)$ ，称为**势**，把每条弧的费用改成

$$\text{约化成本} \quad c'_{uv} = c_{uv} + h(u) - h(v) \quad \text{势是各点的一个数，与走哪条路无关}$$

任何一条从 s 到 t 的路，新费用之和是原来的费用加上 $h(s) - h(t)$ ，中间的项全部抵消。所以**每条 s 到 t 的路都平移同样的量，最短路不变**。这时只要能找到一组势，让所有 $c' \geq 0$ ，就可以在新费用上用 Dijkstra，最后再把距离还原。

这组势怎么找？加一个虚拟源点，向每个点连一条费用 0 的弧，跑一次 Bellman-Ford，取虚拟源到每个点的最短距离作为 h 。最短距离满足 $h(v) \leq h(u) + c_{uv}$ ，这正是 $c' \geq 0$ 。有负环时，这一步就发现了，势不存在。

如果你读过第 5 章，这里应该觉得眼熟。匈牙利算法里，任务价格 u 、车价格 v ，约化成本 $c + v - u \geq 0$ ，等于 0 的边叫紧边，Dijkstra 在约化成本上跑，跑完涨价。那些价格就是这里的势。**势就是网络上的对偶变量**，本章 6.4 会看到它出现在最小费用流的最优性证书里。

在图 6-1 里选“加一条 $c \rightarrow b$ (-4)”和 Johnson，走完全部步骤： $h = (0, 0, -4, 0, -3, 0)$ 之后， $b \rightarrow c$ 、 $b \rightarrow d$ 、 $c \rightarrow b$ 三条弧的新费用都是 0，其余都为正，Dijkstra 在新费用上不会再出错。

什么时候用哪个

费用全部非负：Dijkstra。有负费用但没有负环：Bellman-Ford（单源），或 Johnson（多源，先付一次 Bellman-Ford 的钱）。有负环：先弄清这个环代表什么。物流网络里出现负环，多半说明建模有误，比如把“收益”和“成本”的符号弄反了。

6.3 最大流与最小割

最短路只关心一辆车怎么走。现在换一个问题：**这张网络一个时间窗最多能送多少？**每条弧现在的第一个数是容量：单位时间内最多通过多少辆车、多少托盘。

一个流给每条弧分配一个流量 x_{uv} ，要求 $0 \leq x_{uv} \leq \text{容量}$ ，并且除了 S 和 T 之外，每个点流入等于流出（**流量守恒**）。流量 = 从 S 净流出的量。最大流问题就是让它最大。

增广路与回退弧

从零流开始，反复做一件事：在**残量图**里找一条 S 到 T 的路，沿着它推流。残量图对每条原弧有两个方向：

前向残量

容量 - 已用流量。还能往前推多少。

回退弧

已用流量。可以把已经送出去的流量收回来多少。推流经过回退弧，等于减少原弧的流量。

瓶颈

路径上残量的最小值。沿路推这么多，恰好有一条弧变满（或者一条回退弧被收空）。

回退弧是最大流里最容易被忽略的部分。没有它，贪心地找路推流可能卡在一个不是最大的流上：先推的那条路占了一条关键的弧，别的路就被堵死了。回退弧允许后来的路说“这条弧先借我，我的流量从别处补给你”。Lab 练习 5 里有一个 AI 写的函数就是漏了这一步。

选路的规则也有讲究。随手选一条路（比如深度优先）可能要推很多次；每次选**边数最少**的路（宽度优先），就是 Edmonds-Karp 算法，增广次数不超过 $nm/2$ ，总时间 $O(nm^2)$ ，与容量的大小无关。

先预测 从 S 发出的两条弧容量是 7 和 6，进入 T 的两条弧容量是 6 和 7，合计都是 13。

这张网络的最大流是多少？

13，出弧或入弧被塞满

12

11

10

预测正确。 中间四条弧 $a \rightarrow c$ (4)、 $b \rightarrow c$ (2)、 $b \rightarrow d$ (4)、 $a \rightarrow d$ (2) 把 $\{S, a, b\}$ 与 $\{c, d, T\}$ 隔开，容量之和 12，任何流都要穿过它。13 是一个更松的上界（只看 S 的出弧）。下面的图已经解锁，看最后一步的最小割。

选路规则

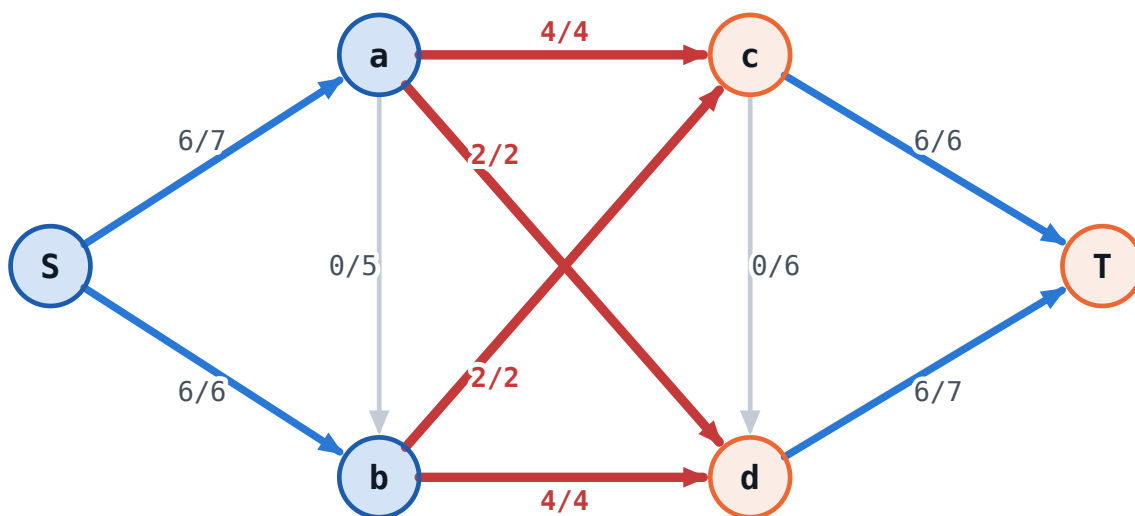
最短增广路 (BFS)

随手选 (DFS)

显示

流量 / 容量

残量：→可推 ←可退



— 有流量 — 这一步的增广路 - - - 用到的回退弧 — 最小割的割边

最短增广路：4 次增广

随手选：8 次增广，其中 4 次用回退弧

#	增广路	瓶颈	累计
1	S→a→c→T	4	4
2	S→a→d→T	2	6
3	S→b→c→T	2	8
4	S→b→d→T	4	12
无增广路，最大流 = 最小割			12

残量图里再没有 S 到 T 的路。从 S 还能到达的点是 {S, a, b} (蓝色)，其余 (橙色) 到不了。

连接两侧的原弧 (红色) 全部饱和：a→c (4)、b→c (2)、b→d (4)、a→d (2)，容量之和 **12** = 流量 **12**。这个割就是证书：任何流都得穿过它，所以不可能超过 12。

图 6-2 最大流。弧上是“流量/容量”。切到“残量”视图，每条弧显示 →还能推多少、←能收回多少。表里点一行可以跳到那一次增广；带↔的行用到了回退弧。最后一步显示最小割。

读图

最短增广路：4 次。先走 $S \rightarrow a \rightarrow c \rightarrow T$ 推 4，再走 $S \rightarrow a \rightarrow d \rightarrow T$ 推 2，然后 $S \rightarrow b \rightarrow c \rightarrow T$ 推 2，最后 $S \rightarrow b \rightarrow d \rightarrow T$ 推 4，一共 12。它们互不干扰，没有用到回退弧。

随手选：8 次，其中一半用到回退弧。深度优先先钻进 $S \rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow T$ 这条弯弯绕绕的路，只推了 2，就把 $b \rightarrow c$ 用满了；后面的路不得不反复把流量退回来重新分配。最终值同样是 12，只是绕了远路。最坏情况下，随手选的次数可以随容量变大而增加，最短增广路的次数只取决于网络的大小。

最大流等于最小割

算法停下来的时候，残量图里已经没有 S 到 T 的路。把从 S 还能到达的点记作 X ，其余记作 Y （包含 T ）。所有从 X 到 Y 的原弧都是满的（否则那里还有残量， Y 里的点就到得了），所有从 Y 到 X 的原弧流量都是 0（否则有回退弧）。于是流量恰好等于 X 到 Y 那些弧的容量之和，这叫**割的容量**。

最大流最小割定理 (Ford-Fulkerson, 1956)

最大流量 = 所有割中最小的容量。

任何流都得穿过任何一个割，所以流量 $\leq$ 割容量，这是“弱”的一半，一行就能证。算法停下来时构造出一个流量等于割容量的流和割，这是“强”的一半。

整数性

容量全是整数时，最大流也可以取整数：每次瓶颈是整数，每次推流都是整数。

这一点在 6.5 节用全单模再证一遍，而且会推广到最小费用流。

图 6-2 的最后一步给出的割是 $\{S, a, b\}$ 对 $\{c, d, T\}$ ，割边是 $a \rightarrow c$ 、 $b \rightarrow c$ 、 $b \rightarrow d$ 、 $a \rightarrow d$ ，容量 $4 + 2 + 4 + 2 = 12$ 。**这个割就是瓶颈的位置**：想让窗口多放行车辆，只有给这四条弧扩容才有用，加宽 $S \rightarrow a$ 或 $d \rightarrow T$ 没有任何效果。做规划时这是一个很实用的结论：最小割告诉你该在哪里投资。

最小割还是最大流的**证书**：拿到一个流和一个割，两个数相等，就不必相信算法，直接检查即可。Lab 练习 2 要求同时返回流和割；练习 5 里有一个 AI 写的函数，流是对的，报出来的割却不是最小的。

6.4 最小费用流

最大流只管送得多。真实问题里每条弧还有单价：走不同的通道费用不同。送 k 个单位，怎么送最便宜？这就是最小费用流，它把最短路和最大流合成了一个问題。

做法直接：**逐次最短路**。每一轮在残量图上找 S 到 T 的最短路（按费用，不按边数），沿它推瓶颈流量。回退弧的费用是原弧费用的相反数，走回退弧等于把之前的一部分流量撤销，还退回它当时花的钱。

回退弧带来负费用，Dijkstra 不能直接用。办法就是 6.2 节的势：维护一组势，让残量图里每条弧的约化成本 $c + h(u) - h(v)$ 都 ≥ 0 ；每一轮 Dijkstra 之后，把每个点的势加上它的最短距离。新增的回退弧恰好落在最短路上，约化成本为 0，所以不变量保持。这 and 第 5 章的“涨价”是同一个动作。

送出的流量 $k = 5$ ☐ 显示势 h

流量 k

5

最大 12

总费用

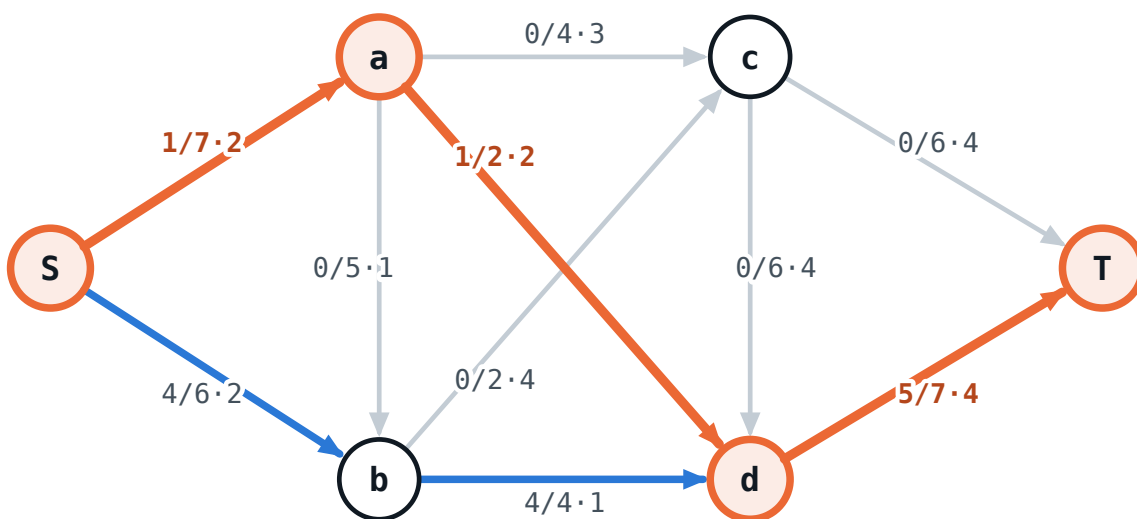
36

逐次最短路

第 k 个单位的价格

8

= 最新一条增广路的长度



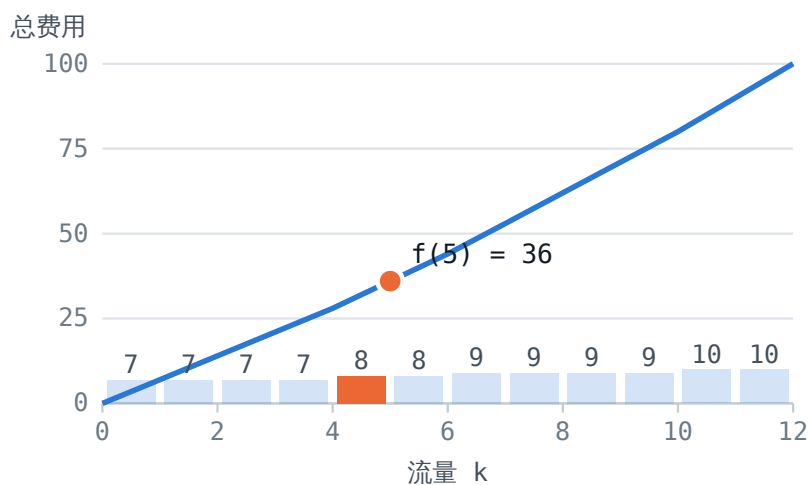
— 有流量 — 刚推的那条最短路

再送第 5 个单位的价格是 **8**：在残量图上找到的最短路长度。

价格比上一个单位涨了：便宜的通道已经满了，只能走更贵的。费用曲线是凸的：边际成本只升不降，这是最短路逐次变长的必然结果。

单纯形 (LP) : 36

LP 的解全是整数



#	最短路	长度	推	累计
1	S→b→d→T	7	4	4
2	S→a→d→T	8	2	6
3	S→a→c→T	9	4	10
4	S→b→c→T	10	2	12

图 6-3 最小费用流。拖动滑块选流量 k ；弧上是“流量/容量·单价”。右图是总费用 $f(k)$ ，柱子是每多送一个单位的价格（边际成本）。表里是逐次最短路四次增广，当前流量落在哪一次就高亮哪一行。右上角的“单纯形（LP）”标签，是把同一个问题交给页面内置的单纯形法，看它是否给出同样的费用，以及解是不是整数。

读图

边际成本逐级上涨：7、7、7、7、8、8、9、9、9、9、10、10。最便宜的通道 S→b→d→T 单价之和是 7，但 b→d 的容量只有 4，四个单位之后它就满了。下一批走 S→a→d→T，单价 8，只有 2 个单位；再下一批走 S→a→c→T，9；最后 S→b→c→T，10。总费用 $4 \times 7 + 2 \times 8 + 4 \times 9 + 2 \times 10 = 100$ 。

费用曲线是凸的。逐次最短路每一轮的路都不比上一轮短：便宜的通道先被用光，剩下的只会更贵。所以 $f(k)$ 是分段线性的凸函数，斜率不降。这对定价有直接含义：送得越多，多送一个单位的成本越高，“平均成本”会低估“边际成本”。

势就是对偶价格。勾选“显示势”，每个点旁边的 h 是当前的势。残量图里所有弧的约化成本都 ≥ 0 ，这就是最优性证书，与第 2 章“对偶变量给出最优性证明”、第 5 章“价格可行且边是紧的”是同一件事。Lab 练习 3 要求返回这组势，由判分程序独立检查。

指派、运输和它们的改派链

第 5 章的指派问题，可以直接写成这张网络的一个特例：

弧	容量	单价	含义
源 → 车 i	1	0	每辆车至多接一个任务
车 i → 任务 j	1	C_{ij}	车 i 去任务 j 的行驶时间
任务 j → 汇	1	0	每个任务恰好一辆车

送 n 个单位的最小费用流，就是最优的指派。逐次最短路每一轮，对应第 5 章“新任务到达时沿一条改派链调整”：残量图里从源出发的最短路，经过“车→任务”的正向弧和“任务→车”的回退弧交替

前进，正是那条改派链；这一轮的路长，就是第 5 章的边际成本。第 5 章的匈牙利算法可以看成把这个通用算法特化到二部图上，价格和势合并成两组。

把中间的容量从 1 改成别的数，得到**运输问题**：多个仓库供货，多个门店需求，每条线路有容量和单价。四向车与提升机的场景里，提升机可以看成一条容量有限的弧（6.7 节会讲怎么处理“点的容量”）。这些问题都能用同一份代码解决。Lab 练习 4 让你把指派写成网络，再调用练习 3 的算法验证 124 s。

代码

下面是 Lab 参考答案里的 `min_cost_flow`，与页面里的 JS 是同一个算法。第 2-11 行建残量图（第 $2k$ 条残量弧是原弧，第 $2k+1$ 条是回退弧），第 13-20 行在有负费用时用 Bellman-Ford 算初始势，主循环（第 22-57 行）每轮先在约化成本上跑一次 Dijkstra（第 23-40 行），再更新势（第 43-45 行）、沿路推流（第 46-57 行）。最后 6 行在最终的残量图上重新算一组势，作为最优性证书。

```

1 def min_cost_flow(n, arcs, s, t, k=None):
2     m = len(arcs)
3     cap = [0] * (2 * m)
4     cost = [0] * (2 * m)
5     adj = [[] for _ in range(n)]
6     for e, (u, v, c, w) in enumerate(arcs):
7         cap[2 * e], cost[2 * e] = c, w
8         cost[2 * e + 1] = -w
9         adj[u].append(2 * e)
10        adj[v].append(2 * e + 1)
11    head = lambda a: arcs[a // 2][1] if a % 2 == 0 else arcs[a // 2][0]
12    pi = [0] * n
13    # 负费用 (无负环) 时先用 Bellman-Ford 算初始势
14    if any(a[3] < 0 for a in arcs):
15        d = [0] * n
16        for _ in range(n):
17            for e, (u, v, _, w) in enumerate(arcs):
18                if d[u] + w < d[v]:
19                    d[v] = d[u] + w
20        pi = d # w + d[u] - d[v] >= 0
21    value = total = 0
22    while k is None or value < k:
23        dist = [INF] * n
24        par = [-1] * n
25        done = [False] * n
26        dist[s] = 0
27        for _ in range(n):
28            u = -1
29            for i in range(n):
30                if not done[i] and dist[i] < INF and (u < 0 or dist[i] < dist[u]):
31                    u = i
32            if u < 0:
33                break
34            done[u] = True
35            for a in adj[u]:
36                v = head(a)
37                if cap[a] > 0 and not done[v]:
38                    nd = dist[u] + cost[a] + pi[u] - pi[v]
39                    if nd < dist[v]:
40                        dist[v], par[v] = nd, a
41        if dist[t] >= INF:
42            break
43        for v in range(n):
44            if dist[v] < INF:
45                pi[v] += dist[v]

```

```

46     path, v = [], t
47     while v != s:
48         path.append(par[v])
49         v = head(par[v] ^ 1)
50     b = min(cap[a] for a in path)
51     if k is not None:
52         b = min(b, k - value)
53     for a in path:
54         cap[a] -= b
55         cap[a ^ 1] += b
56     value += b
57     total += b * sum(cost[a] for a in path)
58     # 证书：在最终残量图上重算一组势（虚拟源 + Bellman-Ford），保证每条残量弧的约化成本 >= 0
59     pi = [0] * n
60     for _ in range(n + 1):
61         for a in range(2 * m):
62             if cap[a] > 0 and pi[head(a ^ 1)] + cost[a] < pi[head(a)]:
63                 pi[head(a)] = pi[head(a ^ 1)] + cost[a]
64     return {"value": value, "cost": total, "flow": [cap[2 * e + 1] for e in range(m)]

```

逐次最短路的增广次数不超过总流量（容量为整数时），所以它是**伪多项式**的：容量特别大时可能很慢。实际系统里更常用的是网络单纯形法和费用缩放法，它们的复杂度只依赖网络的大小。但对本课程的规模（几十到几百个点），逐次最短路已经足够，而且最容易解释。

另一条路：循环取消

先随便造一个满足流量的流（不从零流出发），再检查残量图里有没有**负费用的环**。有的话，沿环推流，总费用严格下降；没有，就是最优。“残量图没有负环”与“存在一组使所有弧约化成本 ≥ 0 的势”是同一个条件的两种说法（6.2 节：有势当且仅当无负环）。Lab 挑战让你实现这个算法，并和逐次最短路对拍。

6.5 网络矩阵为什么不需要分支

前面三节的算法都是组合式的，没有一处用到 LP。可是最大流、最小费用流同样能写成 LP，而且直接给整数解。第 5 章证明过指派矩阵是这样，这一节把同样的证明用到整个网络。

把网络的约束写成矩阵。设网络有 n 个点、 m 条弧，**点弧关联矩阵** A 有 n 行 m 列：弧 (u, v) 对应的那一列，在 u 行是 $+1$ ，在 v 行是 -1 ，其余全是 0 。

流守恒 $Ax = b, 0 \leq x \leq u$ b 在 S 处是 $+k$ ，在 T 处是 $-k$ ，其余是 0

定理

点弧关联矩阵 A 全单模。

证明与第 5 章的引理一模一样，只有三步。

推论（整数性）

容量 u 和流量 k 都是整数，则 $\{x : Ax = b, 0 \leq x \leq u\}$ 的每个顶点都是整数点。

加上单位矩阵（上下界）之后仍全单模。所以最大流、最小费用流、最短路、指派、运输，全都是“LP 就是整数问题”。

- 1 取 A 的任意一个 $k \times k$ 子矩阵 B 。对 k 归纳； $k = 1$ 时元素是 $0, \pm 1$ 。
- 2 若 B 有一列全为 0 ，行列式为 0 。若有一列恰好一个非零元素（ ± 1 ），按这一列展开，行列式是 $\pm$ 一个 $(k-1)$ 阶子式，由归纳假设是 0 或 ± 1 。
- 3 否则 B 的每一列都恰好有两个非零元素。 A 的每一列只有一个 $+1$ 和一个 -1 ，所以 B 的这两个元素也必然是一个 $+1$ 加一个 -1 。把 B 的所有行加起来，得到零向量，行线性相关，行列式为 0 。

整个论证只用了一件事：**每一列恰好一个 $+1$ 和一个 -1** ，也就是“每条弧有一个起点和一个终点”。第 5 章的指派矩阵是二部图的特例，行分两组，每列在两组里各有一个 1 。这里的写法更一般：不需要分组，只需要每列正负各一。

这带来两个实用的结论。第一，**单纯形法不需要分支**：无论是直接用 LP 求解器，还是用 6.4 节的组合算法，得到的最优解都是整数流。第二，**对偶变量也是整数**：费用是整数时，势（对偶变量）也取整数，这解释了为什么本章所有势都是整数，也解释了最大流最小割定理里的割是 $0/1$ 的。

先预测 在原网络上加一行限额：“ $S \rightarrow b$ 与 $b \rightarrow d$ 合计至多 1”，送 1 个单位（原来最便宜的路是 $S \rightarrow b \rightarrow d \rightarrow T$ ，费用 7；次便宜的是 $S \rightarrow a \rightarrow d \rightarrow T$ ，费用 8）。LP 的最优值和整数最优值分别是？

LP 7，整数 7

LP 7.5，整数 8

LP 8，整数 8

LP 7，整数 8

预测正确。 $S \rightarrow b \rightarrow d \rightarrow T$ 会把限额用两次，整数解只好走 8。LP 可以让 $\frac{1}{2}$ 单位走费用 7 的路、 $\frac{1}{2}$ 单位走费用 8 的路，限额合计只用了 $\frac{1}{2} + \frac{1}{2} = 1$ ，费用 7.5。这是一行首尾相接的弧共用限额造成的分数解。下面的检查器已经解锁。

在网络的约束上加

不加

点 c 的容量

串联弧共限额

另一处共限额

送出的流量 $k = 1$



	S	a	a	b	c	c	d	T	T	a
	S	a	a	b	c	c	d	T	T	a
S	1	1	.	.	.	.	.	.	.	.
a	-1	.	1	1	.	.	.	.	.	1
b	.	-1	-1	.	1	1	.	.	.	.
c	.	.	.	-1	-1	.	1	1	.	.
d	.	.	.	.	.	-1	-1	.	1	-1
T	.	.	.	.	.	.	.	-1	-1	.
限额	.	1	.	.	.	1	.	.	.	.

多一行：S→b 与 b→d 合计至多 1。这两条弧首尾相接，走 S→b→d 的一个单位会被计两次。

方程式共 19,447 个，行列式取值 $\{-2, -1, 0, 1, 2\}$ ，其中 $|\det| \geq 2$ 的有 264 个

det = -2 | 136

det = -1 2,223

det = 0 14,636

det = +1 2,324

det = +2 | 128

不是全单模。最小的反例是一个 2×2 的子式，行列式 -2（橙色标出的行和列）：

	S→b	b→d
b	-1	1
限额	1	1

流量 k

1

限额行：S→b + b→d ≤ 1

LP 最优值

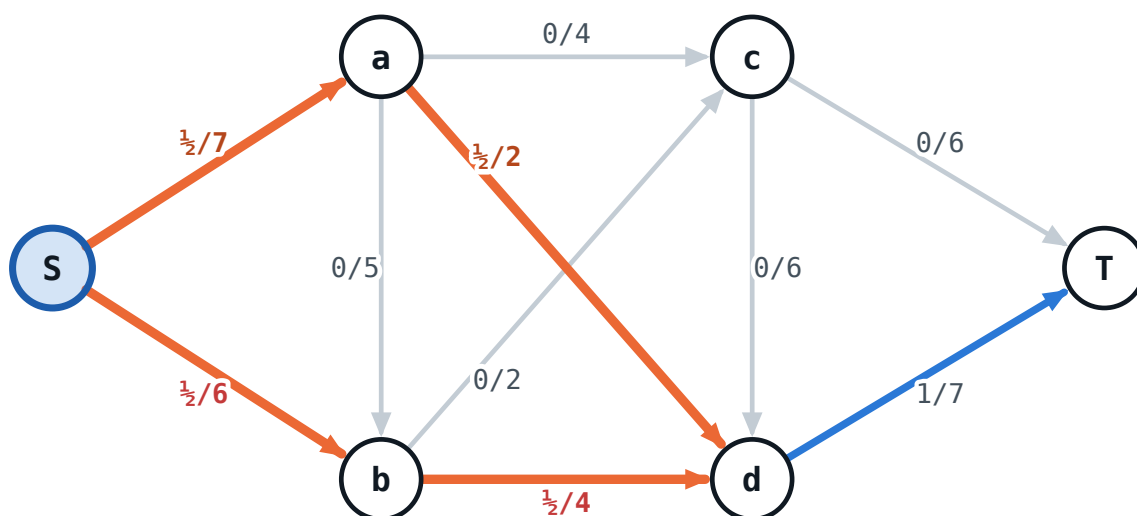
7.5

页面内置单纯形

整数最优值

8

HiGHS (MIP)



LP 的解有分数（橙色的弧）。 LP 值 7.5，整数最优 8，差 0.5。限额行把两条首尾相接的弧绑在一起，一条走 $S \rightarrow b$ 再走 $b \rightarrow d$ 的路占了两次限额；LP 把一个单位劈成两半，各走一半，绕开了这个代价。整数流做不到，只能分支。

图 6-4 网络矩阵的全单模检查。左：点弧关联矩阵（每列 +1、-1 各一个），页面枚举全部方子式，统计行列式的分布；加一行之后再枚举一遍。右：把这张矩阵配上代价，送 k 个单位，看 LP 的最优解（橙色的弧是分数流）和整数最优。LP 用页面内置的单纯形法，整数最优来自 HiGHS 的 MIP。

读图

不加行：6 个点 $\times$ 10 条弧的矩阵，方子式共 8007 个，行列式只有 0、+1、-1 三种，全单模，和证明一致。拖动 k ，LP 的解始终是整数，值与图 6-3 的费用曲线完全一样。

点 c 的容量：“进入 c 的两条弧合计至多 4”。这一行同样是全单模，因为点容量可以通过**拆点**还原成弧容量：把 c 拆成入口和出口，中间加一条容量为 4 的弧。所以点的容量限制不需要担心整数性。提升机一次只能过一辆车，就是这样处理的（6.7 节）。

串联弧共限额：“ $S \rightarrow b$ 与 $b \rightarrow d$ 合计至多 1”。这一行把两条**首尾相接**的弧绑在一起，检查器在多了—行的矩阵里找到行列式 ± 2 的方子式。取 $k = 1$ ：最便宜的路 $S \rightarrow b \rightarrow d \rightarrow T$ （费用 7）会占用这个限额两次，超了；整数解只好走 $S \rightarrow a \rightarrow d \rightarrow T$ ，费用 8。LP 却可以让 $\frac{1}{2}$ 单位走 $S \rightarrow b \rightarrow d \rightarrow T$ 、 $\frac{1}{2}$ 单位走 $S \rightarrow a \rightarrow d \rightarrow T$ ，两条弧合计只用了 $\frac{1}{2} + \frac{1}{2} = 1$ ，费用 7.5。差的 0.5 就是整数性的代价，只有分支才能消除。

另一处共限额是同样的结构，出现在 $k = 7$ 和 8 : LP 53.5，整数最优 54。注意不是每个 k 都会有差距，全单模的意思是“保证”，失去保证不等于每次都出问题。

网络可以加什么，不可以加什么

可以：弧的上下界、点的容量（拆点）、多个源和多个汇（加一个超级源和超级汇）。不可以：把两条不同的弧绑在同一个限额里（尤其是首尾相接的弧）、要求某个点的流入必须来自“同一种”货物。前者破坏“每列一个 $+1$ 一个 -1 ”的结构，后者是下一节的多商品流。

6.6 多商品流：网络被多种货物共用

前面所有问题里，货物都是一种：流到 T 的量不区分是谁的。实际调度里往往有好几辆车、好几种货，每个有自己的起点和终点，却要共用同一张网络的通道。

多商品流：有 K 种商品，商品 k 要从 s_k 送到 t_k 。每种商品各自满足流量守恒，但**所有商品共用弧的容量**：一条弧上各商品的流量之和不超过容量。

单独看每一种商品，它就是一个网络流；合在一起，变量 x_e^k 既出现在自己的守恒行里，又出现在共享的容量行里。一列不再是“一个 $+1$ 一个 -1 ”，整数性的证明失效了。这是问题本身变难了，与写法无关。

LP 的解

自己选整数方案

☐ 只保留商品 1 和 2

LP 最优值

6

每个商品 2 个费用，共 3 个商品

整数最优值

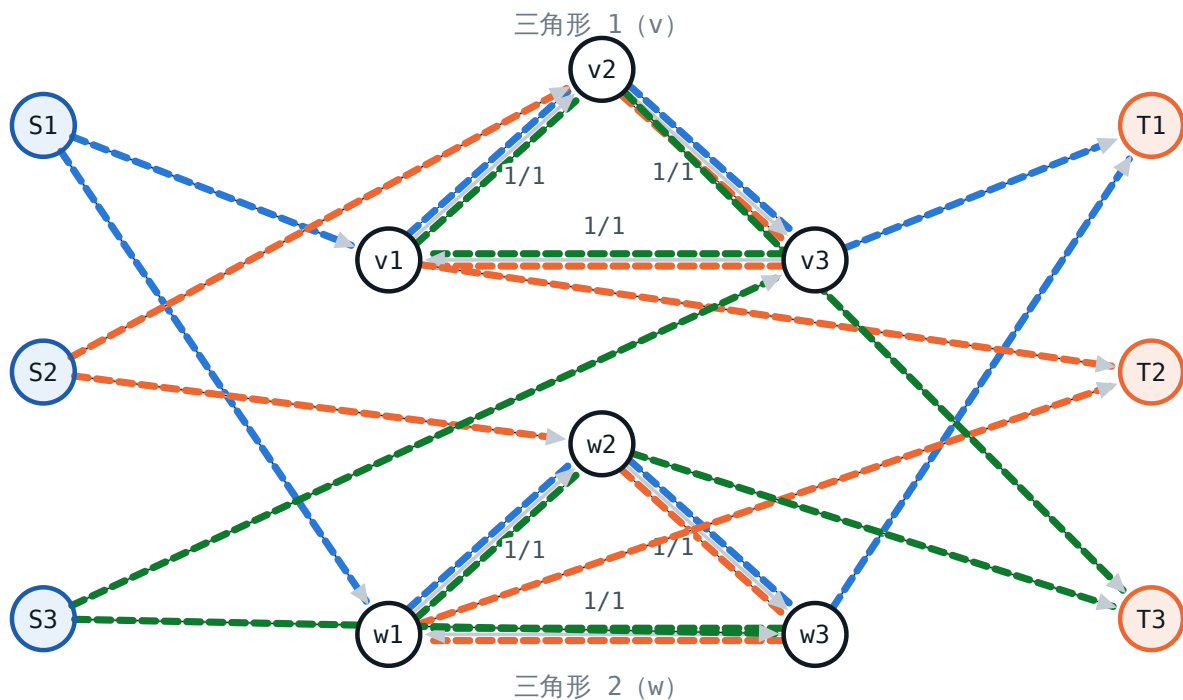
无解

每个商品选一条路

超载的弧

—

LP 里没有超载



商品 1 商品 2 商品 3 虚线 = 每个商品只走 1/2 弧上的数字：载荷/容量

商品 1

三角形 1

三角形 2

商品 2

三角形 1

三角形 2

商品 3

三角形 1

三角形 2

商品 1	商品 2	商品 3	结果
三角形 1	三角形 1	三角形 1	共用一条弧
三角形 2	三角形 1	三角形 1	共用一条弧
三角形 1	三角形 2	三角形 1	共用一条弧
三角形 2	三角形 2	三角形 1	共用一条弧
三角形 1	三角形 1	三角形 2	共用一条弧
三角形 2	三角形 1	三角形 2	共用一条弧
三角形 1	三角形 2	三角形 2	共用一条弧
三角形 2	三角形 2	三角形 2	共用一条弧

LP 的最优解里，每个商品在两个三角形里各走 $\frac{1}{2}$ （虚线）。每条三角形的弧上恰好有两个商品，各占 $\frac{1}{2}$ ，载荷 $1/1$ ，刚好满。

三个商品、两个三角形，每个三角形里任意两个商品的路径都要共用一条弧（每两条路各占两条弧，共三条弧），所以一个三角形里至多容纳一个商品。三个商品放不下，整数无解。LP 却可以把每个商品劈成两半。

图 6-5 三个商品，两个三角形，每条三角形的弧容量为 1。商品 k 从 S_k 进入任一三角形的 v_k （或 w_k ），沿两条弧走到 v_{k+2} （或 w_{k+2} ），再去 T_k 。LP 由页面内置单纯形法求解，整数方案由枚举得到（每个商品的路径只有两条），并用 HiGHS 核对过：LP 值 6，整数无解。

读图

LP 说可以，整数说不行。三个商品各自有两个选择（走三角形 1 或 2），共 8 种整数方案，页面右边的表列出了全部，没有一种可行。原因很短：商品 k 用三角形里的两条弧，商品 $k + 1$ 用另外两条，三条弧里必有一条被两个商品同时占用。所以一个三角形里至多放一个商品，两个三角形至多放两个，三个商品放不下。

LP 可以让每个商品在两个三角形里各走一半。每条三角形的弧上恰有两个商品，各占 $\frac{1}{2}$ ，载荷 1，刚好满。费用 6 就是这个半整数点。勾选“只保留商品 1 和 2”，整数解立即出现（一个商品一个三角形），LP 和整数一样是 4。

这与 5.5 节的“奇圈”是同一个现象：三个东西两两冲突，LP 让每个占一半，整数只能放下其中一部分。三个东西两两冲突，就是图里的一个三角形（奇圈）。

为什么这件事重要

多商品流的整数版本是 NP 难的（Even、Itai、Shamir，1976），LP 松弛通常有分数解，需要分支和割。把它带到第七部分，会看到一个直接的翻译：**每辆车是一种商品**，网络是“位置 $\times$ 时间”展开的时空网络，弧容量和点容量 1 表示同一时刻同一位置只能有一辆车。单辆车自己的最优路线，是时空网络上的一次最短路（6.1 节）；多辆车放在一起，就是一个多商品流，而且要求整数。这就是多智能体路径规划（MAPF）难的根源，也是那一部分用冲突搜索（CBS）而不是一次 LP 的原因。

为什么不能“一辆一辆规划”

如果把每种商品依次当成单商品流来解，后一个商品在前一个商品剩下的容量上走，每一步都是整数、都是最优，但合起来未必最优，甚至可能无解。三个商品的例子里，前两个商品各占一个三角形，第三个商品没有路可走。第 26 章的优先级规划正是这样，它的“不完备”就来自这里。

6.7 在项目里的位置

这一章的三个算法在四向车与提升机的系统里各有位置。判断该用哪个，先问自己：要回答的是“怎么走”、“最多能送多少”，还是“怎么送最省”？

怎么走：最短路

一辆车从当前位置到目标，在栅格图上的最优路线。费用可以是时间、能耗，或者叠加了拥堵惩罚。惩罚为负（“鼓励走某条通道”）时，先检查有没有负环。第 13 章会把它推广到 A* 和栅格图上的启发函数。

最多送多少：最大流与最小割

一个波次的时间窗里，提升机、月台、主巷道各自的容量限制，网络最多放行多少。最小割指出瓶颈在哪里。第 16 章的发运波次要和月台、车辆容量耦合，这一层的估算给了上界。

怎么送最省：最小费用流

多车多任务的分配（第 5 章）、库位间的调拨、多个出入口的分流。任务在到达前后有不同的单价，就是把边际成本曲线（图 6-3）用在决策上。

两个建模技巧

点的容量：拆点。提升机一次只能载一辆车，这是点的限制，而网络流只认弧容量。把点 v 拆成入口 v_{in} 和出口 v_{out} ，中间连一条容量为 1 的弧，所有进入 v 的弧接到 v_{in} ，所有离开 v 的弧从 v_{out} 出发。点容量因此变成弧容量，矩阵仍是全单模（图 6-4 里“点 c 的容量”那一行已经验证过）。Lab 练习 4 里有这个变换。

时间展开：把时间变成网络的一个方向。把网络复制 T 份，每份代表一个时刻，位置 p 在时刻 t 的点记为 (p, t) ；从 (p, t) 到 $(q, t+1)$ 的弧表示这一步从 p 移到 q （要求 p, q 相邻），从 (p, t) 到 $(p, t+1)$ 的弧表示原地等待。之后“在 T 步内把这些货送到”就是这张展开网络上的最大流问题，“最快到达”是最短路。只要不关心具体是哪辆车，它仍然是**单商品**问题，整数性保证成立。一旦要区分车，就是 6.6 节的多商品流。

这个区别对设计很重要：“最多能放行多少辆车”是容易的（多项式，整数），“每辆车走哪条路且互不冲突”是困难的。系统里先用前者估容量、判断窗口是否可行，再用第七部分的方法解后者。

6.8 小结与 Lab

- 1 Dijkstra 靠“确定的点不再改”，这要求费用非负；遇到负费用它不报错，只是悄悄给错答案。Bellman-Ford 每轮松弛全部弧，能处理负费用，第 n 轮还能松弛就是有负环。
- 2 势 h 把费用改成 $c + h(u) - h(v)$ ，所有 s 到 t 的路平移同样的量，最短路不变。Johnson 用它把负费用改成非负；它与第 5 章的价格是同一个东西。
- 3 最大流靠残量图上的增广路，回退弧允许撤销之前的决定；选边数最少的路 (Edmonds-Karp) 时间为 $O(nm^2)$ 。最大流 = 最小割，割是证书，也指出该给哪几条弧扩容。
- 4 最小费用流用逐次最短路加势求解，每一轮是残量图上的一次 Dijkstra。路长单调不降，所以费用曲线是凸的，边际成本等于当前最短路的长度。指派和运输都是它的特例，第 5 章的改派链就是残量图上的增广路。最优性条件：残量图没有负环，也就是存在一组可行的势。
- 5 点弧关联矩阵的每一列有一个 $+1$ 和一个 -1 ，所以全单模。任何整数容量和需求下，LP 的顶点是整数，网络流不需要分支；对偶变量（势）也是整数。
- 6 网络里可以放心加：弧的上下界、点容量（拆点）、超级源汇。不能加：把首尾相接的两条弧绑进同一个限额，这会让 LP 出现分数解，要靠分支切割。
- 7 多商品流让多种货物共用容量，整数性消失，整数版本 NP 难。三个商品、两个三角形的例子里，LP 值 6，整数无解。多辆车的时空规划是它的一个实例，这是第七部分不再指望 LP 的原因。

这一章在你的课题里的位置

派车窗口里的分配用第 5 章的增量匈牙利，它是本章最小费用流的特例。单辆车的路线在时空栅格上求最短路，这是第 13 章 A^* 和第 26 章时空 A^* 的起点。窗口是否装得下，用最大流粗筛，最小割告诉你瓶颈在提升机、主巷道还是月台。

本章的“可以”和“不可以”是一条分界线。分配、单车路径、放行能力都在“可以”一侧，用多项式算法就能精确求解，值得写成独立、可验证的模块。同时考虑多车路线冲突，就跨到了“不可以”一侧，需要下一章开始的分支切割，以及第七部分的专用算法。

术语表

中文	English	在本章的含义
松弛	relaxation	用 $d(u) + c$ 尝试改进 $d(v)$
负环	negative cycle	总费用为负的环，最短路不存在
势、约化成本	potential, reduced cost	每个点的一个数 h ; $c + h(u) - h(v)$ 。最短路不变，用来把负费用改成非负
流、流量守恒	flow, conservation	每条弧的流量；除源汇外每个点流入等于流出
残量图	residual graph	每条弧对应前向残量和回退弧
增广路、瓶颈	augmenting path, bottleneck	残量图里 S 到 T 的路；路上最小的残量
割、最小割	cut, minimum cut	把点分成含 S 和含 T 两侧，割边容量之和；容量最小的割
最大流最小割定理	max-flow min-cut theorem	最大流量 = 最小割的容量
逐次最短路	successive shortest paths	每轮在残量图上找最短路并推流的最小费用流算法
边际成本	marginal cost	多送一个单位的价格，等于当前最短增广路的长度
循环取消	cycle canceling	沿残量图里的负环推流，直到没有负环
点弧关联矩阵	node-arc incidence matrix	每列一个 $+1$ 一个 -1 ，全单模
拆点	node splitting	把点容量变成弧容量
多商品流	multicommodity flow	多种货物共用弧容量，整数版本 NP 难
时间展开网络	time-expanded network	把每个时刻的位置当成一个点，等待和移动是弧

对照教材

- Ahuja、Magnanti、Orlin , *Network Flows: Theory, Algorithms, and Applications* (Prentice Hall , 1993) : 第 4-5 章最短路，第 6-7 章最大流，第 9-10 章最小费用流。本章的算法都在这里，还有证明和复杂度的完整分析。
- Cormen、Leiserson、Rivest、Stein , *Introduction to Algorithms* (第 4 版) : 第 22 章单源最短路 (Dijkstra、Bellman-Ford) ，第 23 章全源最短路 (Johnson) ，第 24 章最大流 (Edmonds-Karp) 。

- Wolsey , *Integer Programming* (第 2 版 , Wiley 2020) : 第 3 章 Well-Solved Problems , 网络矩阵与全单模。
- Bertsimas & Tsitsiklis , *Introduction to Linear Optimization* : 第 7 章 Network flow problems , 含网络单纯形法与整数性。
- Dijkstra , “A note on two problems in connexion with graphs” (1959) ; Bellman , “On a routing problem” (1958) ; Ford , “Network flow theory” (1956) ; Johnson , “Efficient algorithms for shortest paths in sparse networks” (J. ACM , 1977) 。
- Ford & Fulkerson , “Maximal flow through a network” (1956) ; Edmonds & Karp , “Theoretical improvements in algorithmic efficiency for network flow problems” (J. ACM , 1972) ; Dinic (1970) 与 Goldberg & Tarjan 的推进-重标记 (1988) 是更快的最大流算法。
- Hoffman & Kruskal , “Integral boundary points of convex polyhedra” (1956) : 全单模与整数多面体。Even、Itai、Shamir , “On the complexity of timetable and multicommodity flow problems” (SIAM J. Computing , 1976) : 整数多商品流的复杂度。

本章 Lab

代码包 `ch06-network-flow-lab.zip` 随本章一起发放。依赖是 `numpy`、`scipy`、`pytest`；判分用 Floyd 全源最短路、枚举全部割、`linprog` 求得的最小费用，以及对最优性证书的独立检查，不依赖你的代码。

练习	内容	对应
ex1	最短路 ：Dijkstra（遇负费用抛异常）、Bellman-Ford（返回负环）、Johnson；先填 4 个预测	6.1-6.2
ex2	最大流 ：残量图、Edmonds-Karp 与随手选两种选路规则，同时返回最小割	6.3
ex3	最小费用流 ：逐次最短路加势，支持负费用，返回势作为最优性证书；费用曲线	6.4
ex4	把指派写成网络并解出 124 s；点容量的拆点变换	6.4、6.7
ex5	审 5 个 AI 写的最大流函数：不看源码，只靠运行结果归类问题	6.3
挑战	循环取消：检测残量图里的负环，逐步降低费用，与逐次最短路对拍	6.4

```
pip install -r requirements.txt
pytest -q # 全部练习
pytest tests/test_ex3.py -q # 单个练习

# 确认判分本身没问题：参考答案应当全部通过
LAB_TARGET=solutions pytest -q
```

练习 1 要求先在 `PREDICTION` 里写下预测再运行；跑完测试后，终端最后会把你的预测和实际结果并排列出来。

下一章

CHAPTER 07

带容量的指派与背包

本章的网络里，每条弧的容量互不相关，所以 LP 自带整数解。下一章遇到第一个例外：每辆车有载重上限、每个任务占用不同的量（广义指派），或者一个货位只能装下有限的体积（背包）。这时矩阵里出现了不是 0/1 的系数，全单模消失，LP 不再自带整数解，我们回到第 4 章的分支定界和割平面，看背包覆盖割怎样把 LP 拉回整数。

6.9 自测

8 道题，覆盖本章的全部要点。每题选完立即给出解析。

Q1 为什么 **Dijkstra** 在有负费用弧的网络上可能给出错误答案，而且不报错？

它假设一个点一旦被确定，标签就是最终值，以后不再修改；负费用弧可能在之后给这个点更短的路

它不能处理有向图

它只适用于费用为整数的网络

它的复杂度太高

正确。“确定的点不再改”依赖“走一段路只会更长”，也就是费用非负。负费用弧让后面才出现的路更短，Dijkstra 不回头，给出的是一个偏大的数，没有任何提示。

Q2 **Bellman-Ford** 已经做了 $n - 1$ 轮松弛，再做第 n 轮，仍有标签变小。这说明什么？

前面的轮数算错了

从起点可达一个负环

网络不连通

需要换成 Dijkstra

正确。没有负环时，最短路至多 $n - 1$ 条弧， $n - 1$ 轮之后所有标签都是最终值。还能变小，说明有环在越绕越短。沿前驱回走 n 步落在环上，读出来就是证据。

Q3 Johnson 算法里的势 h 满足什么条件，为什么改用 $c' = c + h(u) - h(v)$ 之后最短路不变？

h 是每个点的度数

h 是每个点到终点的距离，它让所有路等长

$h(v) \leq h(u) + c$ ，所以 $c' \geq 0$ ；任何 s 到 t 的路，新费用都恰好比原来多 $h(s) - h(t)$ ，与走哪条路无关

h 全部取 0

正确。 路径上相邻两项的 h 依次抵消，只剩起点和终点。所以每条 s 到 t 的路平移同样的量，最短路不变。用虚拟源加 Bellman-Ford 得到的最短距离满足 $h(v) \leq h(u) + c$ ，保证 $c' \geq 0$ 。

Q4 最大流里的回退弧有什么用？

表示网络里的反向道路

只是记账用，不影响结果

让算法更快

允许把已经送出去的流量收回一部分，改走别处；没有它，贪心地推流可能停在不是最大的流上

正确。 回退弧的残量等于当前的流量。沿它推流就是减少原弧的流量。没有回退弧，先推的路占住的关键弧就永远退不出来，可能停在次优的流上。Lab 练习 5 的 AI-A 就是这样的。

Q5 算法停止后，从 S 在残量图里还能到达的点集是 X 。为什么从 X 到其余点的原弧容量之和恰好等于最大流量？

这些弧一定都是满的（否则残量图里还能走出去），反向弧的流量一定是 0（否则有回退弧），所以流量正好等于这些容量之和

因为 X 里的点是随机选的

因为割不需要满足流量守恒

巧合，由网络的形状决定

正确。 这就是最大流最小割定理的“强”的一半：算法的终止状态本身构造出一个和流量相等的割。任何流都不超过任何割，所以这个割是最小割，这个流是最大流。

Q6 最小费用流的费用曲线 $f(k)$ (送 k 个单位的最小费用) 为什么是凸的?

因为费用总是正的

逐次最短路每一轮的路都不比上一轮短, f 的斜率就是这些路长, 所以斜率不降

因为容量是整数

因为使用了 Dijkstra

正确。 便宜的通道先被用光, 剩下的只会更贵, 边际成本单调不降。这也是“平均成本会低估边际成本”的原因。

Q7 点弧关联矩阵为什么全单模? 关键的一步是?

求解器总是返回整数解

元素都是 0、+1、-1

若一个方子式的每一列都恰有一个 +1 和一个 -1, 所有行相加得零向量, 行线性相关, 行列式为 0; 否则有全零列或单元素列, 按它展开并归纳

任何方阵的行列式都是整数

正确。 关键是每条弧有一个起点和一个终点, 所以每列恰好一个 +1 一个 -1。第 5 章的二部图是它的特例。元素是 0、 ± 1 不足以保证全单模。

Q8 在最小费用流的网络上加下面哪一行, 整数性会被破坏?

“ $S \rightarrow b$ 与 $b \rightarrow d$ 两条首尾相接的弧合计至多 1”

增加一个连到所有源点的超级源

每条弧的流量上界

“进入点 c 的两条弧合计至多 4” (点容量)

正确。 弧的上界是单位矩阵, 点容量可以拆点还原成弧容量, 超级源只是多加了弧, 这三个都保持网络结构。把两条不同的弧绑进同一个限额, 破坏了“每列一个 +1 一个 -1”, 页面找到行列式 ± 2 的子式, LP 出现分数解。

数据说明：本章的全部数字由页面内置的 Dijkstra、Bellman-Ford、Johnson、Edmonds-Karp、逐次最短路、单纯形法和枚举实时计算，并与 Floyd、枚举全部割、SciPy 的 `linprog` 和 HiGHS 的 MIP 逐项对拍（环境：SciPy 1.17.1 · HiGHS 1.12.0）：随机网络上的最短路距离、最大流量与最小割、每个整数流量的最小费用、全单模的行列式计数，都与 Lab 的判分数据同源。